Struktur Pengulangan (Looping)

Naufal Rizqi Habibi
Muhammad Fikri Hidayat

Departemen Matematika
Institut Teknologi Sepuluh Nopember

Week 6

Masalah

Masalah

Melakukan hal yang sama secara berulang kali adalah hal yang membosankan bagi manusia. Penurunan fokus berujung pada kesalahan (*human error*).

Sebaliknya, komputer tidak mengenal rasa bosan. Komputer mampu menjalankan jutaan instruksi yang sama berulang-ulang tanpa kesalahan, asalkan instruksi pengulangan diberikan dengan logika yang tepat.

Solusi: Looping

Di Java, struktur pengulangan utama yang kita pelajari meliputi **for**, **nested loop**, **while**, dan **do-while**.

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Definisi

For digunakan untuk mengeksekusi blok kode berulang kali ketika kita sudah mengetahui **pasti banyaknya perulangan** yang akan dijalankan (*definite loop*).

Listing 1: Sintaks Dasar For

```
for (inisialisasi; kondisi; update) {
    // Blok perintah yang akan diulang
}
```

- **Inisialisasi**: Menyiapkan counter/pencacah awal (dieksekusi sekali di awal).
- **Kondisi**: Ekspresi boolean penentu apakah loop lanjut atau berhenti.
- **Update**: Penambahan (*increment*) atau pengurangan (*decrement*) counter di setiap akhir putaran.

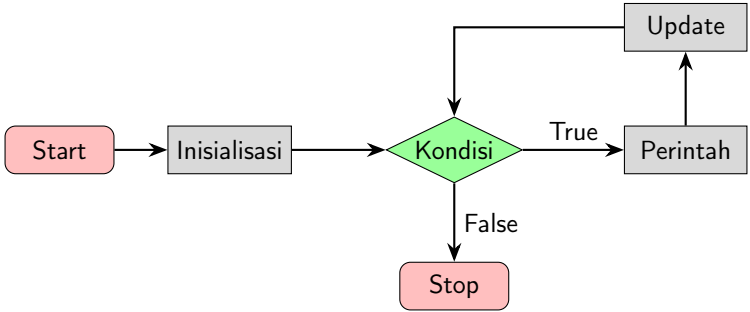


Figure: Alur Eksekusi For Loop

Listing 2: Barisan Aritmatika

```
int suku_n = -2;

for (int n = 1; n <= 10; n++) {
    System.out.print(suku_n + " ");
    suku_n += 3;
}
```

Listing 3: Output

```
-2 1 4 7 10 13 16 19 22 25
```

Penjelasan

Loop dijalankan tepat 10 kali ($n = 1$ hingga $n = 10$). Setiap putaran, nilai `suku_n` bertambah 3.

Instruksi Praktikan

Ketik kode berikut untuk menghitung akumulasi deret: $\sum_{i=1}^{n} i = 1 + 2 + \cdots + n$.

Listing 4: DeretSum.java

```
Scanner in = new Scanner(System.in);
System.out.print("Masukkan nilai n: ");
int n = in.nextInt();

int total = 0;
for (int i = 1; i <= n; i++) {
    total += i;
}
System.out.println("Total deret = " + total)
    ;
```

Listing 5: Output Uji Coba

```
Masukkan nilai n: 5
Total deret = 15
```

Catatan

Variabel akumulasi (`total`) wajib diinisialisasi sebelum loop dimulai agar nilainya tidak ter-reset.

Bahaya Perulangan Tanpa Henti

Jika kondisi terminasi tidak pernah tercapai (`false`), loop akan berputar terus. Program akan mengalami **hang** / **freeze** dan membebani resource CPU hingga dihentikan paksa.

Listing 6: Contoh Infinite Loop (Perhatikan update decrement i– bukan i++)

```
// Nilai i selalu <= 5 karena nilainya terus mengecil (1, 0, -1, -2, ...)
for (int i = 1; i <= 5; i--) {
    System.out.println(i);
}
```

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Nested Loop

Perulangan Bersarang (Loop di dalam Loop)

Definisi

Nested Loop adalah struktur di mana sebuah perulangan berada di dalam badan perulangan lainnya.

- **Outer Loop** (Loop Luar): Mengontrol perulangan tingkat makro (sering dianalogikan sebagai **baris**).
- **Inner Loop** (Loop Dalam): Berjalan hingga tuntas untuk *setiap satu putaran* outer loop (sering dianalogikan sebagai **kolom**).

Listing 7: Sintaks Nested Loop

```
for (int i = 1; i <= baris; i++) {        // Outer Loop
    for (int j = 1; j <= kolom; j++) {    // Inner Loop
        // Perintah yang diulang (baris x kolom kali)
    }
    System.out.println(); // Pindah baris
}
```

Listing 8: PolaSegitiga.java

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("*");
    }
    System.out.println();
}
```

Listing 9: Output

```
*
**
***
****
*****
```

Analisis Iterasi

- Saat $i = 1 \rightarrow j$ jalan 1 kali $\rightarrow$ *
- Saat $i = 2 \rightarrow j$ jalan 2 kali $\rightarrow$ **
- Saat $i = 3 \rightarrow j$ jalan 3 kali $\rightarrow$ ***

Deskripsi Soal

Buat program yang menerima input satu bilangan bulat positif n, lalu cetak pola tangga berisi bilangan genap berurutan menggunakan **nested for loop**.

Hint

- Buat satu variabel counter nilai genap di luar loop (misal `int val = 2`).
- Setiap kali mencetak, lakukan update `val += 2`.
- Dilarang memakai array.

Contoh I/O ($n = 4$)

```
Input :  4
Output :
2
4 6
8 10 12
14 16 18 20
```

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Definisi

While adalah perulangan bertipe *pre-tested loop*. Pengecekan kondisi dilakukan di **awal sebelum** blok perintah dijalankan. Cocok digunakan jika **banyaknya iterasi belum diketahui pasti**.

Listing 10: Sintaks Dasar While

```
while (kondisi) {
    // Blok perintah yang akan diulang
    // Wajib ada perintah update variabel kondisi di dalam sini!
}
```

Karakteristik Pre-Test

Jika kondisi bernilai `false` pada evaluasi pertama, badan loop **tidak akan pernah dijalankan** (0 kali).

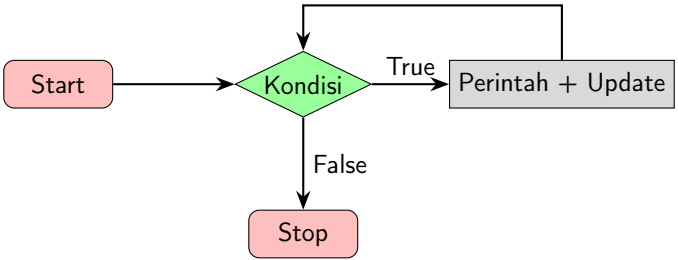


Figure: Alur Eksekusi While Loop

Instruksi Praktikan

Ketik program pembagian bertingkat: Membagi angka input dengan 2 hingga bernilai 0.

Listing 11: BagiDua.java

```
Scanner in = new Scanner(System.in);
System.out.print("Masukkan angka: ");
int angka = in.nextInt();

int step = 0;
while (angka > 0) {
    angka /= 2;
    step++;
    System.out.println("Step " + step + " ->
        " + angka);
}
```

Listing 12: Output Uji Coba

```
Masukkan angka: 8
Step 1 -> 4
Step 2 -> 2
Step 3 -> 1
Step 4 -> 0
```

Penjelasan

Banyaknya iterasi tidak diketahui di awal karena bergantung pada nilai input dinamis.

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Definisi

Do-While adalah struktur pengulangan bertipe *post-tested loop*. Perintah di dalam badan loop akan dieksekusi terlebih dahulu, baru kemudian kondisinya diperiksa di akhir.

Listing 13: Sintaks Dasar Do-While

```
do {
    // Blok perintah (pasti dieksekusi minimal 1 kali)
} while (kondisi);
```

Pembeda Kunci

- Perintah **pasti dieksekusi minimal 1 kali**, meskipun kondisi dari awal bernilai `false`.
- Jangan lupa tanda titik koma (`;`) setelah kurung tutup kondisi.

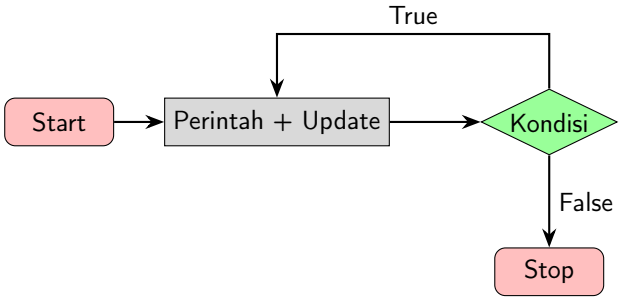


Figure: Alur Eksekusi Do-While Loop

Instruksi Praktikan

Ketik validasi input: Memaksa user input ulang jika angka yang dimasukkan ≤ 0.

Listing 14: ValidasiInput.java

```
Scanner in = new Scanner(System.in);
int input;

do {
    System.out.print("Masukkan angka (> 0):
        ");
    input = in.nextInt();
} while (input <= 0);

System.out.println("Berhasil! Nilai: " +
    input);
```

Listing 15: Output Uji Coba

```
Masukkan angka (> 0): -5
Masukkan angka (> 0): 0
Masukkan angka (> 0): 8
Berhasil! Nilai: 8
```

Mengapa Cocok?

Karena input user harus dieksekusi minimal satu kali sebelum kondisinya divalidasi.

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Struktur	Pengecekan	Min. Eksekusi	Skenario Penggunaan
For	Di awal	0 kali	Banyaknya iterasi sudah pasti/dapat dihitung di awal (*definite*).
While	Di awal	0 kali	Banyaknya iterasi bergantung pada kondisi dinamis / flag boolean.
Do-While	Di akhir	**1 kali**	Minimal satu kali eksekusi wajib dijalankan (misal: menu utama, input validasi).

Daftar isi

1. For Loop
2. Nested Loop
3. While Loop
4. Do-While Loop
5. Ringkasan & Komparasi
6. Tugas Praktikum

Penugasan

Tugas praktikum minggu ini dapat diakses melalui tautan berikut:

Klik di sini untuk membuka lembar soal tugas